Bachelorstudiengang Software Design & Cloud Computing SWD22W (BA, BB)

„Software Architekturen“ | DI Dr. Gaube Gerhard | Sommer 2026

Lab2 | Microservice Architecture with Components

First, we will configure and test components of a microservice architecture. Second, we will orchestrate all these services in a complete architecture.

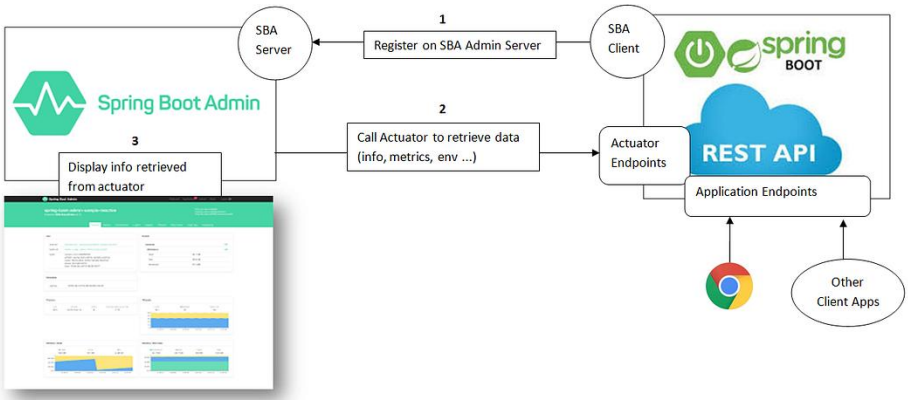
- Task 1: Monitoring Server and Client
- Task 2: Configuration Server
- Task 3: Inter-Service-Communication
- Microservice Architecture
    - Complete architecture `lab2-microservice-archi-final` with User, Product and Order services with H2 DB
    - Uncomplete architecture `lab2-student-service-archi-postgres` with Postgres DB and services to add

Source Code for Lab2: https://github.com/ggaube/swd24-software-architecture

Task 1: Monitoring Server and Client

Spring Boot Admin Server and *Spring Boot Actuator* help to manage application with metric monitoring with **Health Checks Metrics** and **HTTP tracing**. These features can be accessed over HTTP endpoints.

> ***Actuator*** provides metrics to the Spring Boot Admin Server and integrates with external monitoring systems like Prometheus and many more. These systems provide excellent dashboards, graphs, analytics, and alarms to monitor and manage an application from UI.
>
> ***Micrometer*** is used by Actuator, an application metrics framework to integrate with these external application monitoring systems. This makes it easy to plug-in any application monitoring system with little configuration.



Guide: https://spring.io/guides/gs/actuator-service

1.1 Spring Boot Admin Server

1. Setup a microservice with dependencies:
   `spring-boot-admin-starter-server`

2. Set in the main class:
   `@EnableAdminServer`

3. Set the port in the application.properties file
   `server.port = 5000`

4. Start the service and navigate to the Dashboard localhost:5000

1.2 Service Monitoring

1. The client service must be expanded with the Actuator Client dependency:
   `spring-boot-starter-actuator`

2. To find the Monitoring Server and to expose management information, some values have to be added to the `application.properties` file:

```
spring.boot.admin.client.url=http://localhost:5000
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

3. Restart the Service and send a request to these endpoints:
   i. http://localhost:8080/actuator
   ii. http://localhost:8080/actuator/health
   iii. http://localhost:8080/actuator/metrics
   iv. http://localhost:8080/actuator/info

4. Check the Server Dashboard and examine the metrics

Coding Tasks and Control Questions

1. Add another service to your application and enable monitoring
2. Check this metrics provider: https://micrometer.io/ and Cloud Monitoring https://www.dynatrace.com/de/
3. Define the concept of health metrics and health checks
4. Where and how can the expose of actuator endpoints be configured?
5. Test this additional configuration

Task 2: Configuration Server

Spring Boot Configuration Server provides centralized external configuration management for distributed systems, particularly microservices. Instead of each service maintaining its own configuration, the Configuration Server serves as a single source of truth for application configuration.

- **Centralized Configuration**: all configuration files are stored in a central repository, typically Git, but also filesystem, JDBC, …
- **Environment-specific Profiles**: supports multiple profiles (dev, test, prod) to serve environment-specific configurations

Guide: https://spring.io/guides/gs/centralized-configuration

Spring Boot Configuration

1. Setup Config Server with the dependency:
   `spring-cloud-config-server`

2. Enable the server with
   `@EnableConfigServer`

3. Setup the `application.yml` in the git-repository
4. Configure the `application.properties` file in the server and client

Task 3: Synchronous Inter-Service-Communication

A microservice architecture involves multiple microservices, which might communicate with each other. Depending upon business use-cases, this communication can be of synchronous or asynchronous type.

Synchronous inter-service communication with Spring Boot typically involves one service directly making HTTP requests to another service's endpoints. This is often done using libraries like *RestTemplate* provided by Spring.

1. **Sending Requests**: In a client service use a *RestTemplate* or *WebClient* to construct an HTTP request to an endpoint exposed by the server service. This request might include data in the form of query parameters, request body, or headers, depending on the requirements of the endpoint.

2. **Receiving Requests**: In Service B, you would have controllers or handlers that are mapped to specific endpoints. These endpoints define the functionality that Service A can access. When Service B receives a request from Service A, it processes the request, possibly querying a database or performing some business logic, and then generates an appropriate response.

3. **Handling Responses**: Once Service A sends a request to Service B, **it waits synchronously** for a response. When Service B processes the request and generates a response, it sends the response back to Service A. Service A receives the response and can then continue its execution based on the response received.

4. **Error Handling**: Both the client and server services should handle scenarios where requests fail, endpoints are unavailable, or responses contain errors. This can involve handling exceptions, implementing retries, or providing fallback mechanisms.

Coding Tasks

1. Extend the three services by some services and implement the connection
2. Add some error handling code to the services
3. Define these concepts: `RestTemplate, WebClient` fallback mechanisms
4. What can be drawbacks of (many) inter-service calls?
5. Add an arbitrary POST call to the service and test it
6. **BONUS**: Implement following use case: You are visiting an online restaurant homepage, select a meal and click on the “Place Order” button. When this request goes in the *restaurant-service*, the service would want to validate the user details before processing this order. But the user details are managed by the *user-service*. Hence here comes the need to communicate from restaurant-service to user-service.